

Uni-app搜索页面：接口对接与搜索结果预览实现

一、核心需求与接口规范

1.1 功能需求拆解

在已有搜索页面布局基础上，完成“对接搜索接口-获取数据-预览结果”全流程，核心需求包括：

- **接口无缝对接：**封装搜索接口请求，处理请求头、参数传递及token鉴权，确保与后端交互规范。
- **实时结果预览：**输入关键词执行搜索后，快速渲染结果列表，支持结果分类展示（如商品、文章、用户）。
- **状态全面覆盖：**包含加载中、无结果、网络异常、接口报错等状态，提供清晰的用户反馈。
- **结果交互优化：**支持结果项点击跳转、下拉刷新、上拉加载更多，提升预览体验。

1.2 搜索接口规范（与后端约定）

本次对接的搜索接口采用RESTful风格，支持关键词搜索、分页查询，具体规范如下：

参数类型	参数名	说明	示例值
请求参数（GET）	keyword	搜索关键词（必填）	Uni-app开发
	pageNum	当前页码（默认1）	1
	pageSize	每页条数（默认10）	10
请求头	Content-Type	数据格式	application/json
	token	用户登录凭证（可选，用于个性化搜索）	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
响应格式	-	JSON格式，包含分页信息与结果列表	-

1.3 响应数据格式示例

Code block

 $1 \quad \{$

```
2  "code": 200,
3  "msg": "搜索成功",
4  "data": {
5    "total": 28, // 总结果数
6    "pageNum": 1, // 当前页码
7    "pageSize": 10, // 每页条数
8    "pages": 3, // 总页数
9    "list": [
10   {
11     "id": "res001",
12     "type": "article", // 结果类型: article-文章、goods-商品、user-用户
13     "title": "Uni-app搜索功能开发全指南",
14     "cover": "https://api.example.com/covers/search-guide.png",
15     "desc": "详细讲解Uni-app搜索页面布局、接口对接与结果优化",
16     "author": "前端开发组",
17     "createTime": "2025-12-01",
18     "url": "/pages/article/detail?id=res001" // 跳转地址
19   },
20   {
21     "id": "res002",
22     "type": "goods",
23     "title": "Uni-app实战教程书籍",
24     "cover": "https://api.example.com/covers/uni-book.png",
25     "price": 59.9,
26     "sales": 1256,
27     "url": "/pages/goods/detail?id=res002"
28   }
29 ]
30 }
31 }
```

二、接口对接核心实现

2.1 基础准备：请求工具类优化

基于Uni-app封装统一请求工具，支持token自动携带、异常统一处理，文件路径“utils/request.js”：

Code block

```
1  /**
2   * 统一请求工具类
3   * @param {Object} options - 请求配置
4   * @returns {Promise}
5   */
6  export default function request(options) {
```

```
7 // 1. 基础配置
8 const baseURL = process.env.NODE_ENV === 'development'
9   ? 'https://dev-api.example.com'
10   : 'https://api.example.com';
11
12 // 2. 请求参数合并
13 const config = {
14   baseURL,
15   timeout: 10000, // 超时时间10秒
16   header: {
17     'Content-Type': 'application/json',
18     // 自动携带token (登录后存储在本地)
19     'token': uni.getStorageSync('token') || ''
20   },
21   ...options
22 };
23
24 // 3. 请求拦截器 (可选, 如添加签名)
25 // #ifdef APP-PLUS || MP-WEIXIN
26 // 小程序/APP端可添加设备信息等参数
27 config.header.deviceId = uni.getStorageSync('deviceId') || '';
28 // #endif
29
30 // 4. 发起请求
31 return new Promise((resolve, reject) => {
32   uni.request({
33     ...config,
34     success: (res) => {
35       // 5. 响应状态判断 (仅处理200状态码)
36       if (res.statusCode === 200) {
37         const response = res.data;
38         // 6. 业务状态判断
39         if (response.code === 200) {
40           // 业务成功, 返回数据
41           resolve(response.data);
42         } else if (response.code === 401) {
43           // 未登录, 跳转登录页
44           uni.navigateTo({ url: '/pages/login/login' });
45           reject(new Error('请先登录'));
46         } else {
47           // 业务错误, 提示错误信息
48           uni.showToast({ title: response.msg || '操作失败', icon: 'none' });
49           reject(new Error(response.msg || '业务异常'));
50         }
51       } else {
52         // 网络状态错误
53         uni.showToast({ title: `请求失败(${res.statusCode})`, icon: 'none' });
```

```

54         reject(new Error(`HTTP错误: ${res.statusCode}`));
55     }
56 },
57 fail: (err) => {
58     // 7. 请求失败 (网络异常等)
59     uni.showToast({ title: '网络异常, 请检查网络', icon: 'none' });
60     reject(new Error('网络异常'));
61 },
62 complete: () => {
63     // 8. 请求完成 (可关闭加载动画)
64     uni.hideLoading();
65 }
66 });
67 });
68 }

```

2.2 搜索接口封装

封装搜索接口，统一管理请求参数与地址，文件路径“api/search.js”：

Code block

```

1  import request from '@utils/request';
2
3  /**
4   * 执行搜索请求
5   * @param {Object} params - 搜索参数
6   * @param {string} params.keyword - 搜索关键词
7   * @param {number} [params.pageNum=1] - 当前页码
8   * @param {number} [params.pageSize=10] - 每页条数
9   * @returns {Promise}
10  */
11  export const searchResource = (params) => {
12      return request({
13          url: '/search/resource',
14          method: 'GET',
15          params
16      });
17  };
18
19  /**
20   * 获取搜索热词 (可选, 用于热门搜索区动态更新)
21   * @returns {Promise}
22   */
23  export const getSearchHotWords = () => {
24      return request({
25          url: '/search/hot-words',

```

```
26     method: 'GET'
27   });
28   };
```

2.3 页面中接口调用逻辑

在搜索页面中集成接口调用，处理搜索触发、分页加载、结果更新等核心逻辑，完善“pages/search/search.vue”：

Code block

```
1  <script>
2  import { throttle } from '@utils/tools';
3  import { searchResource, getSearchHotWords } from '@api/search';
4
5  export default {
6    name: 'SearchPage',
7    data() {
8      return {
9        searchValue: '', // 搜索输入值
10       historyList: [], // 搜索历史
11       hotList: [], // 热门搜索（动态从接口获取）
12       suggestList: [], // 搜索建议
13
14       // 搜索结果相关
15       resultData: {
16         list: [], // 结果列表
17         total: 0, // 总结果数
18         pageNum: 1, // 当前页码
19         pageSize: 10, // 每页条数
20         pages: 0 // 总页数
21       },
22       showResult: false, // 是否显示结果区
23       loading: false, // 加载状态（防止重复请求）
24       isLastPage: false // 是否为最后一页
25     };
26   },
27   onLoad() {
28     // 页面加载：读取历史+获取热门搜索
29     this.getSearchHistory();
30     this.fetchHotWords();
31     // 开启下拉刷新
32     uni.startPullDownRefresh();
33   },
34   onPullDownRefresh() {
35     // 下拉刷新：重置页码，重新搜索
```

```
36     if (this.searchValue.trim()) {
37         this.resultData.pageNum = 1;
38         this.fetchSearchResult(true);
39     } else {
40         uni.stopPullDownRefresh();
41     }
42 },
43 onReachBottom() {
44     // 上拉加载更多
45     if (this.showResult && !this.loading && !this.isLastPage) {
46         this.resultData.pageNum += 1;
47         this.fetchSearchResult(false);
48     }
49 },
50 methods: {
51     // 节流处理输入事件
52     handleInput: throttle(function() {
53         const value = this.searchValue.trim();
54         if (value) {
55             this.showResult = false;
56             // 匹配历史生成建议（可扩展为接口建议）
57             this.suggestList = this.historyList.filter(item =>
58                 item.toLowerCase().includes(value.toLowerCase())
59             );
60         } else {
61             this.suggestList = [];
62             this.showResult = false;
63         }
64     }, 500),
65
66     // 从接口获取热门搜索词
67     async fetchHotWords() {
68         try {
69             const res = await getSearchHotWords();
70             this.hotList = res || ['Uni-app', 'mp-html', '搜索接口'];
71         } catch (err) {
72             console.error('获取热门搜索失败: ', err);
73             // 接口失败时使用默认热词
74             this.hotList = ['Uni-app', 'mp-html', '搜索接口'];
75         }
76     },
77
78     // 执行搜索：调用接口获取结果
79     async handleSearch() {
80         const keyword = this.searchValue.trim();
81         if (!keyword) {
82             uni.showToast({ title: '请输入搜索关键词', icon: 'none' });
```

```
83         return;
84     }
85     // 保存搜索历史
86     this.saveSearchHistory(keyword);
87     // 重置页码，获取第一页结果
88     this.resultData.pageNum = 1;
```

三、搜索结果预览与渲染

3.1 结果分类渲染模板

根据接口返回的“type”字段（article/goods/user），实现分类渲染，适配不同类型结果的展示需求：

Code block

```
1  <!-- 搜索结果区：分类渲染 -->
2  <view class="search-result" v-if="showResult">
3
4      <view class="result-count" v-if="resultData.total > 0">
5          找到 {{ resultData.total }} 条关于“{{ searchValue }}”的结果
6      </view>
7
8
9      <view class="result-empty" v-if="resultData.total === 0">
10         <image src="/static/empty-search.png" class="empty-icon" mode="widthFix">
11             </image>
12         <view class="empty-text">未找到相关结果</view>
13         <view class="recommend-text">为你推荐</view>
14         <view class="recommend-list">
15             <view class="recommend-item"
16                 v-for="(item, index) in hotList"
17                 :key="index"
18                 @click="selectKeyword(item)">
19                 {{ item }}
20             </view>
21         </view>
22
23
24         <view class="result-list" v-if="resultData.total > 0">
25
26             <view class="result-group" v-if="resultData.list.some(item => item.type
27                 === 'article')">
28                 <view class="group-title">相关文章</view>
29                 <view class="article-item"
```

```
29         v-for="(item, index) in resultData.list.filter(i => i.type ===
'article'))"
30         :key="item.id"
31         @click="navigateTo(item.url)">
32         <image :src="item.cover" class="article-cover" mode="aspectFill">
</image>
33         <view class="article-info">
34             <view class="article-title">{{ item.title }}</view>
35             <view class="article-desc">{{ item.desc }}</view>
36             <view class="article-meta">
37                 <view class="author">{{ item.author }}</view>
38                 <view class="time">{{ item.createTime }}</view>
39             </view>
40         </view>
41     </view>
42 </view>
43
44
45     <view class="result-group" v-if="resultData.list.some(item => item.type
=== 'goods')">
46         <view class="group-title">相关商品</view>
47         <view class="goods-list">
48             <view class="goods-item"
49                 v-for="(item, index) in resultData.list.filter(i => i.type ===
'goods')">
50                 :key="item.id"
51                 @click="navigateTo(item.url)">
52                 <image :src="item.cover" class="goods-cover" mode="aspectFill">
</image>
53                 <view class="goods-title">{{ item.title }}</view>
54                 <view class="goods-price">¥{{ item.price.toFixed(2) }}</view>
55                 <view class="goods-sales">已售 {{ item.sales }}</view>
56             </view>
57         </view>
58     </view>
59
60
61 </view>
62
63
64     <view class="load-more" v-if="showResult && resultData.total > 0">
65         <uni-load-more
66             :status="isLastPage ? 'noMore' : (loading ? 'loading' : 'more')"
67             :contentText="{ noMore: '没有更多结果了' }">
68         />
69     </view>
70 </view>
```


3.2 结果项样式设计 (scoped)

Code block

```
1  /* 结果统计 */
2  .result-count {
3      font-size: 24rpx;
4      color: #999;
5      padding: 16rpx 0;
6      border-bottom: 1px solid #f5f5f5;
7      margin-bottom: 20rpx;
8  }
9
10 /* 结果分组标题 */
11 .group-title {
12     font-size: 28rpx;
13     font-weight: 500;
14     color: #333;
15     margin-bottom: 20rpx;
16     padding-left: 12rpx;
17     border-left: 4rpx solid #007aff;
18 }
19
20 /* 文章结果项 */
21 .article-item {
22     display: flex;
23     margin-bottom: 24rpx;
24     background-color: #fff;
25     border-radius: 16rpx;
26     overflow: hidden;
27     box-shadow: 0 1px 3px rgba(0, 0, 0, 0.05);
28 }
29 .article-cover {
30     width: 200rpx;
31     height: 140rpx;
32     flex-shrink: 0;
33 }
34 .article-info {
35     flex: 1;
36     padding: 20rpx;
37 }
38 .article-title {
39     font-size: 28rpx;
40     color: #333;
41     margin-bottom: 12rpx;
42     line-height: 1.4;
```

```
43     display: -webkit-box;
44     -webkit-line-clamp: 2;
45     -webkit-box-orient: vertical;
46     overflow: hidden;
47 }
48 .article-desc {
49     font-size: 24rpx;
50     color: #999;
51     margin-bottom: 16rpx;
52     line-height: 1.3;
53     display: -webkit-box;
54     -webkit-line-clamp: 1;
55     -webkit-box-orient: vertical;
56     overflow: hidden;
57 }
58 .article-meta {
59     display: flex;
60     justify-content: space-between;
61     font-size: 22rpx;
62     color: #ccc;
63 }
64
65 /* 商品结果项 */
66 .goods-list {
67     display: flex;
68     flex-wrap: wrap;
69     gap: 20rpx;
70     margin-bottom: 24rpx;
71 }
72 .goods-item {
73     width: calc(50% - 10rpx);
74     background-color: #fff;
75     border-radius: 16rpx;
76     padding: 20rpx;
77     box-shadow: 0 1px 3px rgba(0, 0, 0, 0.05);
78 }
79 .goods-cover {
80     width: 100%;
81     height: 240rpx;
82     border-radius: 8rpx;
83     margin-bottom: 16rpx;
84 }
85 .goods-title {
86     font-size: 26rpx;
87     color: #333;
88     margin-bottom: 12rpx;
89     line-height: 1.3;
```

3.3 结果项跳转逻辑

根据结果项的“url”字段实现页面跳转，支持不同类型结果跳转到对应详情页：

Code block

```
1  // 结果项跳转
2  navigateTo(url) {
3    if (!url) return;
4    // 判断跳转方式: navigateTo/redirectTo/tabBar
5    if (url.includes('/pages/tabBar/')) {
6      // 跳转到tabBar页面
7      uni.switchTab({ url });
8    } else {
9      // 跳转到普通页面
10     uni.navigateTo({ url });
11   }
12 }
```

四、关键优化与异常处理

4.1 性能优化策略

- **防止重复请求：**通过“loading”状态控制，避免同一时间多次调用接口（如上拉加载时快速滑动）。
- **分页加载：**采用分页查询减少单次请求数据量，提升首屏加载速度，支持下拉刷新与上拉加载。
- **数据缓存：**搜索历史本地缓存，热门搜索接口数据可添加短期缓存（如5分钟），减少重复请求。
- **图片优化：**结果项图片使用“aspectFill”模式避免变形，可结合Uni-app图片懒加载（需在pages.json配置）：

Code block

```
1  // pages.json中配置图片懒加载
2  {
3    "pages": [
4      {
5        "path": "pages/search/search",
6        "style": {
7          "enablePullDownRefresh": true,
8          "imageLazyLoad": true // 开启图片懒加载
9        }
10     }
11   ]
12 }
```

4.2 异常场景全覆盖

异常场景	处理方案	用户反馈
未输入关键词搜索	拦截搜索请求，提示用户输入关键词	Toast提示“请输入搜索关键词”
网络异常	请求工具类统一捕获，回滚页码	Toast提示“网络异常，请检查网络”
接口超时	设置10秒超时时间，触发fail回调	Toast提示“请求超时，请重试”
未登录（401）	请求拦截器跳转至登录页	自动跳转，登录后可返回搜索页
搜索无结果	展示空状态，推荐热门关键词	显示空图标+“未找到相关结果”，提供推荐词点击
上拉加载失败	回滚当前页码，允许再次尝试	Toast提示错误信息，加载状态重置

4.3 多端适配细节

- **导航栏适配：**在App端与小程序端，搜索头部固定定位时，需预留状态栏高度，可通过Uni-app API获取：

Code block

```
1  // 在onLoad中获取状态栏高度，用于适配
2  onLoad() {
3      const systemInfo = uni.getSystemInfoSync();
4      this.statusBarHeight = systemInfo.statusBarHeight;
5      // 其他初始化逻辑
6  }
```

Code block

```
1  /* 适配状态栏高度 */
2  .search-header {
3      padding-top: calc(var(--status-bar-height) + 16rpx);
4      padding-bottom: 16rpx;
5      /* 原有样式 */
6  }
```

- **下拉刷新样式：**在pages.json中配置下拉刷新颜色，适配品牌风格：

Code block

```
1  // pages.json
2  {
3    "globalStyle": {
4      "pullDownRefresh": {
5        "color": "#007aff", // 刷新图标颜色
6        "backgroundColor": "#f5f5f5" // 刷新区域背景色
7      }
8    }
9  }
```

五、扩展功能与进阶方向

5.1 搜索建议接口对接

将现有“历史匹配建议”升级为“接口实时建议”，提升建议精准度：

Code block

```
1  // 接口封装：获取搜索建议
2  export const getSearchSuggest = (keyword) => {
3    return request({
4      url: '/search/suggest',
5      method: 'GET',
6      params: { keyword }
7    });
8  };
9
10 // 页面中调用（替换原历史匹配逻辑）
11 handleInput: throttle(async function() {
12   const value = this.searchValue.trim();
13   if (value) {
14     this.showResult = false;
15     try {
16       // 调用接口获取建议
17       const res = await getSearchSuggest(value);
18       this.suggestList = res || [];
19     } catch (err) {
20       // 接口失败时 fallback 到历史匹配
21       this.suggestList = this.historyList.filter(item =>
22         item.includes(value)
23       );
24     }
25   }
26 });
```

```
24     }
25   } else {
26     this.suggestList = [];
27   }
28 }, 500);
```

5.2 结果筛选与排序

在结果区顶部添加筛选栏，支持按类型（文章/商品）、时间、热度排序，请求时携带筛选参数：

Code block

```
1  // 筛选参数
2  data() {
3    return {
4      // ...其他数据
5      filterParams: {
6        type: 'all', // all-全部, article-文章, goods-商品
7        sort: 'relevance' // relevance-相关度, time-时间, hot-热度
8      }
9    };
10 },
11
12 // 搜索接口参数追加筛选条件
13 const params = {
14   keyword,
15   pageNum: this.resultData.pageNum,
16   pageSize: this.resultData.pageSize,
17   type: this.filterParams.type,
18   sort: this.filterParams.sort
19 };
```

5.3 搜索埋点与数据分析

添加搜索行为埋点，统计关键词热度、搜索转化率等数据，用于优化搜索体验：

Code block

```
1  // 埋点：记录搜索行为
2  trackSearchBehavior(keyword) {
3    // 调用埋点接口
4    this.$api.track({
5      event: 'search',
6      keyword,
7      timestamp: Date.now(),
8      page: 'search-page'
```

```
9    });  
10   },  
11  
12   // 在handleSearch中调用  
13   async handleSearch() {  
14     const keyword = this.searchValue.trim();  
15     if (!keyword) return;  
16     // 埋点记录  
17     this.trackSearchBehavior(keyword);  
18     // 其他逻辑...  
19   }
```

六、总结

本次实现完成了搜索页面从“接口对接”到“结果预览”的全流程开发，核心亮点包括：

- **规范的接口交互：**封装统一请求工具，支持鉴权、超时处理，与后端交互符合RESTful规范。
- **流畅的结果预览：**分类渲染不同类型结果，支持下拉刷新、上拉加载，适配多端展示需求。
- **全面的异常处理：**覆盖网络异常、接口报错、无结果等场景，提供清晰的用户反馈。
- **可扩展的架构：**预留筛选、排序、精准建议等扩展点，便于后续功能迭代。

开发过程中需重点关注接口请求的安全性（如token携带）、性能优化（如分页与懒加载）及多端适配细节，确保搜索功能在不同设备与网络环境下均能稳定、高效运行。

（注：文档部分内容可能由 AI 生成）